

Bash — Niveau avancé

Partie 2 — Cahier d'exercices avancés (≈ 3 heures)

Regex & parsing · Descripteurs & here-docs · Sous-shells & coproc ·

Fonctions avancées · Robustesse (trap/getopts) · Concurrency · IPC

Prérequis : avoir terminé le premier cahier d'exercices (navigation, redirections, variables, structures de contrôle, tableaux, arithmétique de base). Ce cahier va plus loin : robustesse des scripts, communication entre processus, expansions avancées et petits outils en ligne de commande complets. Les corrigés se trouvent en fin de document.

Sommaire

- 1. Regex avancées & parsing de texte (25 min)
- 2. Here-docs, here-strings & descripteurs de fichiers (20 min)
- 3. Sous-shells, coproc & substitution de processus (28 min)
- 4. Fonctions avancées : récursion, variadique, nameref (32 min)
- 5. Tableaux avancés (10 min)
- 6. Robustesse : set -euo pipefail, trap, getopts (41 min)
- 7. Arithmétique bit à bit (10 min)
- 8. Agrégation de données réelles (14 min)
- 9. Concurrency, jobs & signaux (22 min)
- 10. Communication inter-processus : mkfifo (14 min)
- 11. Projets de synthèse avancés (38 min)
- Corrigés

1. Regex avancées & parsing de texte

On dépasse ici grep simple : groupes de capture, classes POSIX, et les limites du découpage naïf par séparateur.

Exercice 30 — Parsing d'un log avec BASH_REMATCH

■ 12 min · Difficulté : difficile

Voici une ligne de log de type serveur web :

```
192.168.1.12 - - [17/Sep/2026:10:22:31] "GET /api/users HTTP/1.1" 200 512
```

Écrivez un test `[[ $ligne =~ regex ]]` avec des groupes de capture () qui extrait, dans des variables séparées via `BASH_REMATCH` : l'adresse IP, la méthode HTTP (GET/POST/...), le chemin demandé, et le code de statut. Affichez ces quatre valeurs.

Faites en sorte que le script rejette (message d'erreur) toute ligne dont le code de statut ne fait pas exactement 3 chiffres.

Exercice 31 — CSV avec champs entre guillemets — les limites de cut

■ 12 min · Difficulté : difficile

Soit la ligne CSV suivante, où un champ contient une virgule protégée par des guillemets :

```
Dupont,"Paris, France",Ingénieur
```

Montrez que `cut -d " " " découpe incorrectement cette ligne (le champ ville est coupé en deux). Écrivez ensuite une boucle while IFS=, read -r a b c adaptée qui, sans installer d'outil externe, gère correctement ce cas précis en utilisant une expression régulière avec =~ plutôt qu'un simple découpage par virgule.`

Concluez en une phrase : pour un vrai parseur CSV robuste (échappement, guillemets imbriqués), pourquoi Bash seul n'est-il pas le bon outil ?

2. Here-docs, here-strings & descripteurs de fichiers

Manipuler l'entrée/sortie avec précision : here-docs, here-strings, et duplication de descripteurs avec exec.

Exercice 32 — Here-doc vs here-string, expansion ou non

■ 8 min · Difficulté : moyen

Avec `nom="Alice"`, écrivez :

- un here-doc (`<<EOF`) qui affiche "Bonjour \$nom" avec expansion de la variable
- le même here-doc mais avec le délimiteur entre guillemets (`<<'EOF'`), et observez la différence
- une here-string (`<<<`) qui envoie directement "\$nom" en entrée de la commande `wc -c`

Expliquez en une phrase pourquoi la version avec guillemets n'affiche pas la valeur de la variable.

Exercice 33 — exec et duplication de descripteurs

■ 12 min · Difficulté : difficile

Écrivez un script qui :

- sauvegarde le descripteur de sortie standard actuel dans le descripteur 6 (`exec 6>&1`)
- redirige ensuite toute la sortie standard du script vers un fichier `rapport.log` avec `exec > rapport.log`
- affiche plusieurs lignes avec `echo` (elles doivent aller dans le fichier, pas à l'écran)
- restaure la sortie standard d'origine à partir du descripteur 6, puis affiche un message de confirmation qui doit lui apparaître à l'écran

3. Sous-shells, coproc & substitution de processus

Comprendre où vivent vos variables, et comment faire dialoguer Bash avec d'autres processus sans fichier temporaire.

Exercice 34 — Portée des variables : sous-shell vs bloc courant

■ 8 min · Difficulté : moyen

Initialisez `compteur=0`. Exécutez :

```
( compteur=99 )
echo "$compteur"

{ compteur=99; }
echo "$compteur"
```

Expliquez la différence de résultat entre les deux blocs. Donnez un cas réel où cette différence provoque un bug classique (indice : une variable modifiée à l'intérieur d'un pipeline).

Exercice 35 — coproc — dialoguer avec un processus

■ 15 min · Difficulté : expert

Lancez `bc -l` (calculatrice) comme coprocess avec `coprocess` via `coproc`. Écrivez ensuite dans son entrée (`${COPROC[1]}`) l'expression `3.5*2`, lisez le résultat depuis sa sortie (`${COPROC[0]}`), et affichez-le. Envoyez une deuxième expression au même processus pour prouver qu'il reste actif entre les deux échanges.

Exercice 36 — Substitution de processus

■ 8 min · Difficulté : moyen

Vous avez deux commandes qui produisent chacune une liste triée de noms (par exemple `sort fichier1.txt` et `sort fichier2.txt`). Comparez leurs sorties avec `diff` sans créer aucun fichier intermédiaire, en utilisant la substitution de processus `<( )`.

Exercice 37 — mapfile / readarray

■ 8 min · Difficulté : moyen

Chargez le contenu d'un fichier texte de plusieurs lignes directement dans un tableau avec `mapfile` (sans boucle `while read`). Affichez le nombre de lignes chargées, puis affichez uniquement les lignes non vides, en supprimant le retour à la ligne final de chaque élément.

4. Fonctions avancées

Récursion, arguments variables, portée locale, et passage par référence.

Exercice 38 — Récursion vs itération

■ 10 min · Difficulté : moyen

Écrivez une fonction récursive `factorielle n` qui retourne (via `echo` et capture par `$( )`) la factorielle de `n`. Écrivez ensuite une version itérative de Fibonacci `fibo n` qui calcule le `n`-ième terme sans récursion (pour éviter l'explosion du nombre d'appels). Testez `factorielle 6` et `fibo 10`.

Exercice 39 — Fonction variadique et code de retour

■ 12 min · Difficulté : difficile

Écrivez une fonction moyenne qui accepte un nombre quelconque d'entiers en arguments, les stocke dans une variable `local`, et affiche leur moyenne arrondie à l'entier. La fonction doit retourner un code 1 (échec) si elle est appelée sans aucun argument, et afficher un message d'erreur sur la sortie d'erreur standard dans ce cas, sans faire planter le script appelant.

Exercice 40 — Passage par référence avec `declare -n`

■ 12 min · Difficulté : expert

Écrivez une fonction `ajouter_element` qui prend le nom d'un tableau et une valeur, et ajoute la valeur à la fin de ce tableau — en modifiant directement le tableau de l'appelant (pas une copie), grâce à une référence nommée (`declare -n`).

Testez avec un tableau `panier=("pomme")` : après `ajouter_element panier "poire"`, `panier` doit contenir deux éléments.

5. Tableaux avancés

Exercice 41 — Fréquence de mots avec tableau associatif

■ 10 min · Difficulté : difficile

Étant donné la phrase `texte="le chat mange le poisson le chat dort"`, utilisez un tableau associatif pour compter les occurrences de chaque mot, puis affichez le résultat trié par nombre d'occurrences décroissant, sous la forme "mot: n".

Indice : découpez la phrase en mots avec `read -a`, puis triez la sortie finale avec `sort` sur le bon champ.

6. Robustesse : `set -euo pipefail`, `trap`, `getopts`

Les réflexes qui distinguent un script fragile d'un outil fiable en production.

Exercice 42 — Comprendre `pipefail`

■ 12 min · Difficulté : difficile

Exécutez `false | echo "ok"` et affichez `$?` : le code obtenu est-il celui de `false` ou de `echo` ?

Activez `set -o pipefail` et recommencez. Expliquez la différence, puis écrivez un pipeline `grep introuvable fichier.txt | wc -l` et montrez comment `pipefail` change la détection d'échec quand `grep` ne trouve rien.

Exercice 43 — `trap EXIT / ERR`, `mktemp` et `$LINENO`

■ 14 min · Difficulté : expert

Écrivez un script qui :

- active `set -euo pipefail`
- crée un fichier temporaire avec `mktemp` et y écrit des données
- installe un `trap ... EXIT` qui supprime ce fichier temporaire quoi qu'il arrive (succès, erreur, ou interruption)
- installe un `trap ... ERR` qui affiche "Erreur ligne `$LINENO`" avant que le script ne s'arrête
- provoque volontairement une erreur (par exemple lire un fichier inexistant) pour vérifier que les deux traps se déclenchent bien, dans le bon ordre

Exercice 44 — `getopts` — mini interface en ligne de commande

■ 15 min · Difficulté : expert

Écrivez un script `outil.sh` qui accepte les options suivantes via `getopts` :

- `-v` : active un mode verbeux (variable `VERBOSE=1`)
- `-o fichier` : indique un fichier de sortie (argument obligatoire)
- `-h` : affiche l'aide et quitte avec le code 0

Le script doit rejeter une option inconnue avec un message d'erreur explicite et un code de sortie non nul, et afficher, en mode verbeux uniquement, la valeur du fichier de sortie choisi (ou `"stdout"` par défaut si `-o` n'a pas été fourni).

7. Arithmétique bit à bit

Exercice 45 — Masques de permissions avec les opérateurs binaires

■ 10 min · Difficulté : difficile

En travaillant uniquement avec `$( ( ))` et les opérateurs `& | ^ ~ << >>`, à partir de la valeur octale de permission `0o640` :

- extrayez uniquement les 3 bits du propriétaire (indice : masque `0700` puis décalage)
- ajoutez le droit d'exécution pour le propriétaire avec un OU binaire
- retirez le droit de lecture pour le groupe avec un AND et une négation
- affichez le résultat final en octal avec `printf "%o\n"`

8. Agrégation de données réelles

Exercice 46 — Agrégation horaire de logs

■ 14 min · Difficulté : expert

Réutilisez le format de log de l'exercice 30 (plusieurs lignes, différentes heures). Écrivez un script qui lit le fichier ligne par ligne, extrait l'heure (HH) de chaque timestamp avec une regex, compte le nombre de requêtes par heure dans un tableau associatif, puis affiche l'heure ayant le plus grand nombre de requêtes ainsi que son total.

9. Concurrency, jobs & signaux

Exercice 47 — `wait -n`, `timeout` et parallélisation

■ 12 min · Difficulté : difficile

Lancez trois commandes `sleep` de durées différentes (2s, 4s, 6s) en arrière-plan. Utilisez `wait -n` dans une boucle pour afficher un message dès qu'une tâche se termine, dans l'ordre où elles finissent réellement (pas dans l'ordre de lancement).

Ajoutez ensuite une quatrième tâche protégée par `timeout 3 sleep 10` : vérifiez son code de retour et affichez si elle a été interrompue pour dépassement de délai.

Exercice 48 — Gestion fine des signaux

■ 10 min · Difficulté : difficile

Écrivez un script longue durée (boucle infinie avec `sleep 1`) qui :

- affiche "Arrêt propre demandé" et se termine correctement en recevant `SIGTERM`

- affiche "Interruption clavier" et se termine en recevant SIGINT
- ignore complètement SIGHUP (le script continue de tourner)

Testez-le en lui envoyant les signaux depuis un autre terminal avec `kill -SIGNAL pid`.

10. Communication inter-processus : mkfifo

Exercice 49 — Producteur / consommateur avec un tube nommé

■ 14 min · Difficulté : expert

Créez un tube nommé avec `mkfifo`. Dans un premier terminal (ou en lançant un processus en arrière-plan), faites tourner une boucle qui lit en continu ce tube et affiche chaque message reçu préfixé de "reçu: ". Depuis un second terminal (ou une seconde commande), écrivez plusieurs messages dans le tube à quelques secondes d'intervalle et vérifiez qu'ils apparaissent bien côté lecteur. Nettoyez le tube à la fin avec `rm`.

11. Projets de synthèse avancés

Exercice 50 — Superviseur de logs robuste (CLI complet)

■ 20 min · Difficulté : expert

Écrivez un script `supervise.sh` qui rassemble la plupart des notions du cahier :

- utilise `getopts` pour accepter `-d dossier` (obligatoire) et `-s seuil` (nombre d'erreurs à partir duquel alerter, défaut 5)
- active `set -euo pipefail` et un `trap ERR` qui affiche la ligne fautive
- parcourt tous les fichiers `*.log` du dossier, et pour chacun compte le nombre de lignes contenant "ERROR" dans un tableau associatif indexé par nom de fichier
- affiche un tableau récapitulatif trié par nombre d'erreurs décroissant
- si un fichier dépasse le seuil, l'archive individuellement dans `alertes/nomfichier_$(date +%Y%m%d_%H%M%S).tar.gz` (créant le dossier si besoin)
- termine avec un code de sortie égal au nombre de fichiers ayant dépassé le seuil (0 si aucun)

Exercice 51 — File d'attente de tâches avec verrou

■ 18 min · Difficulté : expert

Simulez une file d'attente de tâches partagée entre plusieurs exécutions concurrentes du même script, en évitant que deux instances traitent la même tâche en même temps :

- un fichier `taches.txt` contient une tâche par ligne
- le script retire et traite une seule tâche à la fois (par exemple `echo "traitement: $tache" puis sleep 1 pour simuler du travail), en protégeant l'accès au fichier avec un verrou atomique — utilisez mkdir (atomique sur la plupart des systèmes de fichiers) comme mécanisme de verrou nommé taches.lock, avec une boucle d'attente si le verrou est déjà pris`
- libère systématiquement le verrou à la fin, y compris en cas d'erreur (`indice : trap ... EXIT`)

Lancez le script deux fois en parallèle (`./file.sh & ./file.sh &`) et vérifiez qu'aucune tâche n'est traitée deux fois.

Corrigés

Solutions proposées — d'autres approches correctes existent, en particulier pour les projets de synthèse.

Exercice 30 — Parsing d'un log avec BASH_REMATCH

```
ligne='192.168.1.12 - - [17/Sep/2026:10:22:31] "GET /api/users HTTP/1.1" 200 512'
regex='^([0-9.]+) .* "([A-Z]+) ([^ ]+) HTTP/[0-9.]+ "([0-9]+) '

if [[ "$ligne" =~ $regex ]]; then
    ip="${BASH_REMATCH[1]}"
    methode="${BASH_REMATCH[2]}"
    chemin="${BASH_REMATCH[3]}"
    statut="${BASH_REMATCH[4]}"
    if [[ ! "$statut" =~ ^[0-9]{3}$ ]]; then
        echo "Code de statut invalide" >&2; exit 1
    fi
    echo "IP=$ip METHODE=$methode CHEMIN=$chemin STATUT=$statut"
fi
```

Exercice 31 — CSV avec champs entre guillemets

```
ligne='Dupont,"Paris, France",Ingénieur'

# cut découpe naïvement sur toutes les virgules :
cut -d "," -f2 <<< "$ligne" # -> "Paris (tronqué, faux)

# Avec une regex adaptée à CE format précis (un seul champ entre guillemets) :
regex='^([^\,]+),("[^"]+"),(.+)$'
if [[ "$ligne" =~ $regex ]]; then
    nom="${BASH_REMATCH[1]}"; ville="${BASH_REMATCH[2]}"; metier="${BASH_REMATCH[3]}"
    echo "nom=$nom ville=$ville metier=$metier"
fi
# Bash n'a pas de vrai parseur CSV (guillemets échappés, retours ligne dans un
# champ...) : au-delà d'un cas simple, awk/python/csvkit sont plus fiables.
```

Exercice 32 — Here-doc vs here-string, expansion ou non

```
nom="Alice"

cat <<EOF
Bonjour $nom
EOF

cat <<'EOF'
Bonjour $nom
EOF
# affiche littéralement "Bonjour $nom" : le délimiteur entre guillemets
# désactive toute expansion à l'intérieur du here-doc.

wc -c <<< "$nom"■
```

Exercice 33 — exec et duplication de descripteurs

```
#!/bin/bash
exec 6>&1
exec > rapport.log

echo "Ligne 1 du rapport"
echo "Ligne 2 du rapport"

exec 1>&6 6>&-
echo "Rapport terminé, voir rapport.log"■
```

Exercice 34 — Portée des variables : sous-shell vs bloc courant

```
compteur=0
( compteur=99 )
echo "$compteur"           # 0 : le sous-shell a sa propre copie

{ compteur=99; }
echo "$compteur"           # 99 : { } s'exécute dans le shell courant

# Piège classique : "cat f | while read l; do ((n++)); done" exécute le while
# dans un sous-shell (à cause du pipe) : $n reste à 0 après la boucle.
```

Exercice 35 — coproc — dialoguer avec un processus

```
coproc BC { bc -l; }
echo "3.5*2" >&"${BC[1]}"
read -r resultat <&"${BC[0]}"
echo "Résultat: $resultat"

echo "10/4" >&"${BC[1]}"
read -r resultat2 <&"${BC[0]}"
echo "Résultat: $resultat2"

exec {BC[1]}>&-
```

Exercice 36 — Substitution de processus

```
diff <(sort fichier1.txt) <(sort fichier2.txt)
```

Exercice 37 — mapfile / readarray

```
mapfile -t lignes < fichier.txt
echo "Nombre de lignes: ${#lignes[@]}"
for l in "${lignes[@]}"; do
    [[ -n "$l" ]] && echo "$l"
done
```

Exercice 38 — Récursion vs itération

```
factorielle() {
    local n=$1
    if (( n <= 1 )); then
        echo 1
    else
        echo $(( n * $(factorielle $((n-1))) ))
    fi
}
echo "6! = $(factorielle 6)"

fibonacci() {
    local n=$1 a=0 b=1
    for (( i=0; i<n; i++ )); do
        local tmp=$((a+b)); a=$b; b=$tmp
    done
    echo "$a"
}
echo "fibonacci(10) = $(fibonacci 10)"■
```

Exercice 39 — Fonction variadique et code de retour

```

moyenne() {
    if [[ $# -eq 0 ]]; then
        echo "moyenne: aucun argument fourni" >&2
        return 1
    fi
    local total=0
    for n in "$@"; do
        (( total += n ))
    done
    echo $(( total / $# ))
    return 0
}

moyenne 10 15 20    # affiche 15
moyenne || echo "échec attendu, code $?"■

```

Exercice 40 — Passage par référence avec declare -n

```

ajouter_element() {
    local -n tableau_ref="$1"
    tableau_ref+=("$2")
}

panier=("pomme")
ajouter_element panier "poire"
echo "${panier[@]}"      # pomme poire
echo "${#panier[@]}"     # 2

```

Exercice 41 — Fréquence de mots avec tableau associatif

```

texte="le chat mange le poisson le chat dort"
declare -A freq
read -ra mots <<< "$texte"
for m in "${mots[@]}"; do
    ((freq[$m]++))
done

for m in "${!freq[@]}"; do
    echo "$m: ${freq[$m]}"
done | sort -t: -k2 -nr

```

Exercice 42 — Comprendre pipefail

```

false | echo "ok"
echo $?      # 0 : code de la dernière commande du pipeline (echo)

set -o pipefail
false | echo "ok"
echo $?      # 1 : pipefail remonte l'échec de "false"

grep introuvable fichier.txt | wc -l
echo $?      # sans pipefail : 0 (celui de wc) même si grep ne trouve rien
              # avec pipefail : 1 (grep ne trouve rien = échec)

```

Exercice 43 — trap EXIT / ERR, mktemp et \$LINENO

```
#!/bin/bash
set -euo pipefail

tmpfile=$(mktemp)
echo "données temporaires" > "$tmpfile"

trap 'rm -f "$tmpfile"' EXIT
trap 'echo "Erreur ligne $LINENO"' ERR

cat "$tmpfile"
cat fichier_qui_nexiste_pas # déclenche le trap ERR, puis le trap EXIT nettoie
```

Exercice 44 — getopt — mini interface en ligne de commande

```
#!/bin/bash
VERBOSE=0
OUTPUT="stdout"

usage() { echo "Usage: $0 [-v] [-o fichier] [-h]"; exit 0; }

while getopt "vo:h" opt; do
    case $opt in
        v) VERBOSE=1 ;;
        o) OUTPUT="$OPTARG" ;;
        h) usage ;;
        \?) echo "Option invalide: -$OPTARG" >&2; exit 1 ;;
    esac
done

if [[ $VERBOSE -eq 1 ]]; then
    echo "Sortie configurée sur: $OUTPUT"
fi
```

Exercice 45 — Masques de permissions avec les opérateurs binaires

```
perm=0640

proprietaire=$(( (perm & 0700) >> 6 ))
echo "$proprietaire" # 6

perm=$(( perm | 0100 )) # ajoute +x pour le propriétaire -> 0740
perm=$(( perm & ~0040 )) # retire le read du groupe -> 0700

printf "%o\n" "$perm"■
```

Exercice 46 — Agrégation horaire de logs

```
declare -A par_heure

while IFS= read -r ligne; do
    if [[ "$ligne" =~ \[[0-9]+\/[A-Za-z]+\/[0-9]+:\{0-9\}\{2\}: ]]; then
        heure="${BASH_REMATCH[1]}"
        ((par_heure[$heure]++))
    fi
done < acces.log

max_heure=""; max_val=0
for h in "${!par_heure[@]}"; do
    if (( par_heure[$h] > max_val )); then
        max_val=${par_heure[$h]}; max_heure=$h
    fi
done
echo "Heure la plus active: ${max_heure}h ($max_val requêtes)"■
```

Exercice 47 — wait -n, timeout et parallélisation

```
sleep 2 & sleep 4 & sleep 6 &
while (( $(jobs -r | wc -l) > 0 )); do
    wait -n
    echo "Une tâche vient de se terminer"
done

if timeout 3 sleep 10; then
    echo "Terminé normalement"
else
    echo "Interrompu pour dépassement de délai (code $?)"
fi
```

Exercice 48 — Gestion fine des signaux

```
#!/bin/bash
trap 'echo "Arrêt propre demandé"; exit 0' SIGTERM
trap 'echo "Interruption clavier"; exit 0' SIGINT
trap '' SIGHUP

while true; do
    sleep 1
done
# depuis un autre terminal : kill -SIGTERM <pid> / kill -SIGINT <pid> / kill -SIGHUP <pid>
```

Exercice 49 — Producteur / consommateur avec un tube nommé

```
mkfifo /tmp/montube

# côté lecteur (à lancer en premier, en arrière-plan ou dans un autre terminal)
while read -r msg; do
    echo "reçu: $msg"
done < /tmp/montube &

# côté écrivain
echo "message 1" > /tmp/montube
sleep 2
echo "message 2" > /tmp/montube

rm /tmp/montube
```

Exercice 50 — Superviseur de logs robuste (CLI complet)

```
#!/bin/bash
set -euo pipefail
trap 'echo "Erreur ligne $LINENO" >&2' ERR

SEUIL=5
while getopts "d:s:" opt; do
    case $opt in
        d) DOSSIER="$OPTARG" ;;
        s) SEUIL="$OPTARG" ;;
        \?) echo "Option invalide" >&2; exit 1 ;;
    esac
done
: "${DOSSIER:?Erreur: -d dossier est obligatoire}"

declare -A erreurs
for f in "$DOSSIER"/*.log; do
    [[ -e "$f" ]] || continue
    n=$(grep -c "ERROR" "$f" || true)
    erreurs["$(basename "$f")"]=$n
done

for f in "${!erreurs[@]}"; do
    echo "$f: ${erreurs[$f]}"
done | sort -t: -k2 -nr

nb_alertes=0
mkdir -p alertes
for f in "${!erreurs[@]}"; do
    if (( erreurs[$f] > SEUIL )); then
        tar -czf "alertes/${f}_${date +%Y%m%d_%H%M%S}.tar.gz" "$DOSSIER/$f"
        ((nb_alertes++))
    fi
done
exit "$nb_alertes"
```

Exercice 51 — File d'attente de tâches avec verrou

```
#!/bin/bash
LOCK="taches.lock"

acquerir_verrou() {
    while ! mkdir "$LOCK" 2>/dev/null; do
        sleep 0.2
    done
}
liberer_verrou() { rmdir "$LOCK" 2>/dev/null || true; }
trap liberer_verrou EXIT

acquerir_verrou
tache=$(head -n1 taches.txt 2>/dev/null || true)
if [[ -n "$tache" ]]; then
    sed -i '1d' taches.txt
fi
liberer_verrou
trap - EXIT

if [[ -n "$tache" ]]; then
    echo "traitement: $tache"
    sleep 1
else
    echo "file vide"
fi
```